

Wstęp do programowania obiektowego

Bartosz Miąskowski

Zmienne

Zmienna (*variable*) – nazwana lokalizacja w pamięci, która przechowuje dane określonego **typu**. Działa jako symboliczna nazwa dla wartości, którą może reprezentować, umożliwiając programiście manipulowanie tymi danymi w programie.

```
int wiek = 18;  
string imie = "Jan";  
  
wiek = 19; // zmiana wartości
```

Definicja a deklaracja

```
int liczba;      // deklaracja
liczba = 5;      // przypisanie (inicjalizacja)
int wiek = 20;   // deklaracja z przypisaniem
```

- **Deklaracja** – poinformowanie kompilatora, że istnieje zmienna o danym typie i nazwie.
- **Definicja (inicjalizacja)** – nadanie jej wartości początkowej.

Zasięg zmiennych

Zasięg (scope) określa, **gdzie w kodzie** zmienna jest widoczna i może być używana.

```
void Test()  
{  
    int x = 10; // lokalna zmienna  
  
    if (x > 0)  
    {  
        int y = 20; // widoczna tylko w tym bloku  
        Console.WriteLine(y);  
    }  
  
    // Console.WriteLine(y); // błąd! y nie istnieje poza blokiem  
}
```

Zmienna znika po zakończeniu swojego bloku kodu.

Stałe

Stała (*constant*) – nazwana lokalizacja w pamięci, której **wartość musi być ustalona w momencie kompilacji** i która **nie może ulec zmianie** podczas całego cyklu życia programu.

Stałe są domyślnie statyczne.



```
const double Pi = 3.14159;
```

Paradygmaty

Paradygmaty programowania

Definicja

Paradygmat to sposób myślenia o programowaniu – zestaw zasad, które określają **jak tworzymy programy**, **jak organizujemy kod** i **rozwiązujemy problemy**.

Paradygmat możemy porównać do:

- **Filozofii programowania** – czyli jak postrzegasz / rozumiesz świat kodu.
- **Stylu pisania** – tak jak w języku naturalnym można pisać wiersze, opowiadania lub raporty, tak w programowaniu można pisać kod w oparciu o różne paradygmaty.

Paradygmaty programowania

Co obejmuje paradygmat?

- Sposób opisu danych i operacji.
- Sposób organizacji programu.
- Poziom abstrakcji – czy skupiamy się na tym „*jak coś zrobić*”, czy „*co ma być zrobione*”.
- Rola programisty

Programowanie imperatywne

Paradygmat **imperatywny** polega na **wydawaniu komputerowi poleceń krok po kroku** — mówimy *jak* coś ma zostać zrobione.

Idea:

Program to **ciąg instrukcji**, które **zmieniają stan pamięci**. Każda instrukcja coś robi — przypisuje wartość, porównuje, wykonuje skok, wywołuje funkcję itd.

Cechy charakterystyczne:

- Skupienie się krok po kroku na tym, co trzeba zrobić.
- Wykorzystuje zmienne i instrukcje sterujące (*if, for, while, switch*).
- Programista zarządza przepływem sterowania.
- Kod odzwierciedla kolejność działań komputera.

Programowanie imperatywne

```
#include <stdio.h>

int main() {
    int suma = 0;

    for (int i = 1; i <= 5; i++) {
        suma += i; // modyfikujemy stan zmiennej
    }

    printf("Suma: %d\n", suma);
    return 0;
}
```

C jako język imperatywny

Tutaj program:

1. Tworzy zmienną suma.
2. Wykonuje pętlę krok po kroku.
3. Za każdym razem zmienia jej wartość.
4. Na końcu wypisuje wynik.

To klasyczny przykład imperatywnego podejścia: program opisuje *dokładnie* jak dojść do wyniku.

*Paradygmat imperatywny to jak przepis dla kucharza: „Najpierw posiekaj cebulę, potem ją podsmaż, dodaj przyprawy...”
– każdy krok musi być opisany w odpowiedniej kolejności.*

Programowanie funkcyjne

Paradygmat **funkcyjny** polega na **opisywaniu obliczeń za pomocą funkcji matematycznych**, a nie na wydawaniu poleceń krok po kroku. **Programista mówi co ma być obliczone, a nie jak to zrobić.**

Idea:

Program składa się z **czystych funkcji** (*pure functions*), które:

- zawsze zwracają ten sam wynik dla tych samych danych wejściowych,
- nie zmieniają stanu programu (*brak efektów ubocznych*),
- nie modyfikują zmiennych (*bo wszystko jest **niemutowalne***).

Cechy charakterystyczne:

- Brak lub ograniczenie mutacji (*zmienne są niezienne – **let** zamiast **var***).
- Brak stanów globalnych i efektów ubocznych.
- Funkcje wyższego rzędu – przyjmują inne funkcje jako argumenty.
- Rekurencja zamiast pętli.
- Deklaratywny styl – mówisz co ma być obliczone, nie jak krok po kroku.

Programowanie funkcyjne

Tu nie ma żadnych pętli, liczników ani zmiennych, które się zmieniają.

Po prostu deklarujemy **co chcemy zrobić**:
„zsumuj elementy listy”.

```
let liczby = [1; 2; 3; 4; 5]

// sumujemy wszystkie elementy listy
let suma = List.sum liczby

printfn "Suma: %d" suma
```

F# jako język funkcyjny

Programowanie funkcyjne



```
//Czysta funkcja w F# (tak wygląda)  
let dodaj a b = a + b
```

F# jako język funkcyjny

Ta funkcja:

- Nie zmienia niczego poza sobą.
- Nie zależy od żadnych zewnętrznych danych.
- Zawsze zwróci ten sam wynik dla tych samych argumentów.

Programowanie funkcyjne to jak **matematyka w kodzie** — zamiast mówić „zrób to, potem to”, mówisz „wynik to suma tej listy”.

Programowanie funkcyjne a imperatywne

```
int suma = 0;  
for (int i = 1; i <= 5; i++) suma += i;
```

C jako język imperatywny

```
//W F# nie ma tradycyjnej pętli  
let suma = [1..5] |> List.sum
```

F# jako język funkcyjny

Programowanie logiczne

Paradygmat **logiczny** polega na tym, że programista **nie pisze algorytmu**, tylko **opisuje fakty i reguły**, a komputer sam **wnioskuje** (*logicznie dedukuje*) rozwiązanie.

Idea:

Program to baza wiedzy:

- **fakty** — mówią, co jest prawdą,
- **reguły** — określają zależności między faktami,
- **zapytania** — pytamy system: „czy coś jest prawdą?” lub „co spełnia dane warunki?”.

Programowanie logiczne

Przykładowe fakty i reguły:

```
rodzic(jan, anna).  
rodzic(anna, kasia).  
  
dziadek(X, Y) :- rodzic(X, Z), rodzic(Z, Y).
```

Prolog jako język logiczny

Zapytanie:

```
?- dziadek(jan, kasia).
```

Prolog jako język logiczny | wynik: **true**

Na przedstawionym przykładzie program nie wykonuje żadnych kroków – **on myśli**:

- Jan jest rodzicem Anny.
- Anna jest rodzicem Kasi.
- Więc Jan jest dziadkiem Kasi.

Cechy charakterystyczne:

- Program to **zbiór reguł i faktów**, nie instrukcji.
- Nie ma tradycyjnych zmiennych – są **symbole logiczne**.
- **Wnioskowanie automatyczne** – program sam szuka rozwiązania.
- Paradygmat oparty o **logikę predykatów**.
- Styl – *co jest prawdą, nie jak to zrobić*.

Programowanie logiczne to jak **rozwiązywanie zagadki detektywistycznej** – podajesz fakty i reguły, a komputer sam odkrywa, *kto jest winny*.

Programowanie deklaratywne

Paradygmat **deklaratywny** polega na **opisywaniu tego, co chcemy uzyskać**, zamiast mówić komputerowi, *jak* ma to zrobić.

Idea:

- Opisujemy cel, nie kroki.
- Komputer sam wybiera sposób wykonania.
- Nie interesuje nas algorytm – **tylko wynik**.

Programowanie deklaratywne

```
SELECT imie, nazwisko  
FROM Uczniowie  
WHERE klasa = '4A'  
ORDER BY nazwisko;
```

SQL jako język deklaratywny

Tutaj nie piszesz *jak* przeszukiwać bazę danych — po prostu mówisz **co chcesz otrzymać**:

„Pokaż imię i nazwisko uczniów z klasy 4A, posortowanych alfabetycznie.”

Sposób realizacji (indeksy, optymalizacja, plan zapytania) **robi za Ciebie silnik SQL**.

Programowanie deklaratywne

Tu nie mówisz *jak* przeglądarka ma narysować nagłówek, tylko **co ma się na stronie znaleźć**.

Resztę — układ, renderowanie, style — **robi silnik przeglądarki**.

Cechy charakterystyczne:

- Opisujesz cel, nie proces.
- Brak instrukcji sterujących (if, for, while).
- Programy są łatwe do czytania, trudniejsze do optymalizacji ręcznie.
- Wiele języków dziedzinowych (DSL) korzysta z tego podejścia — np. SQL, HTML, CSS, **XAML**, **LINQ**.

Programowanie deklaratywne to jak **zamówienie w restauracji**: mówisz co chcesz zjeść, a nie jak kucharz ma to przygotować.



```
<h1>Witaj na mojej stronie</h1>  
<p>To jest przykładowy akapit tekstu.</p>
```

HTML jako język deklaratywny

Programowanie obiektowe

OOP (*object oriented programming*) – paradygmat, w którym **program składa się z obiektów**, czyli połączenia **danych (pól)** i **zachowań (metod)**.

Obiekt to **element, który łączy w sobie stan (dane) i zachowanie (metody)**. Jest to podejście modelujące rzeczywiste byty i zjawiska jako niezależne, współpracujące ze sobą elementy programu.

Filary OPP

- **Enkapsulacja** (*hermetyzacja*) – ukrywanie wewnętrznych szczegółów implementacji obiektu i chronienie danych przed nieautoryzowanym dostępem. Obiekt udostępnia jedynie niezbędny interfejs do interakcji.
- **Abstrakcja** – skupianie się na istotnych cechach obiektu, z pominięciem zbędnych szczegółów. Tworzenie prostego modelu, który ukrywa skomplikowaną logikę pod spodem.
- **Dziedziczenie** (*inheritance*) – mechanizm pozwalający jednej klasie (**klasa potomna**) przejąć (*odziedziczyć*) właściwości i metody innej klasy (**klasa bazowa**), promując ponowne użycie kodu.
- **Polimorfizm** (*wielopostaciowość*) – zdolność obiektów różnych klas do odpowiadania na to samo wywołanie (*np. tę samą nazwę metody*) w różny, specyficzny dla siebie sposób.

Programowanie obiektowe

```
class Samochod
{
    public string Marka;
    public void Jedz()
    {
        Console.WriteLine($"{Marka} jedzie!");
    }
}

var auto = new Samochod();
auto.Marka = "BMW";
auto.Jedz(); // BMW jedzie!
```

C# jako język obiektowy

Programowanie obiektowe opiera się na dwóch podstawowych pojęciach:

- **Klasa** – szablon lub plan, który definiuje właściwości (*dane/attributy*) i zachowania (*metody*), jakie będą miały obiekty utworzone na jej podstawie.
- **Obiekt** – instancja (*konkretny egzemplarz*) klasy, posiadająca własny, unikalny stan (*konkretne wartości atrybutów*).

Programowanie obiektowe to jak **projektowanie świata w miniaturze** – tworzysz obiekty, które mają własne dane i potrafią coś robić.

Paradygmaty programowania – porównanie

Cecha / Paradygmat	Imperatywny	Deklaratywny	Funkcyjny	Logiczny	Obiektowy (OOP)
Sposób myślenia	Opisuje <i>jak</i> coś zrobić, krok po kroku	Opisuje <i>co</i> ma być zrobione	Opisuje <i>co obliczyć</i> za pomocą funkcji	Opisuje <i>co jest prawdą</i> na podstawie faktów i reguł	Opisuje świat w postaci obiektów z danymi i zachowaniem
Kontrola przepływu	Jawna (pętle, instrukcje <code>if</code> , <code>while</code>)	Ukryta – wykonanie kontroluje silnik lub interpreter	Rekurencja i funkcje zamiast pętli	Silnik logiczny przeszukuje fakty	Obiekty komunikują się metodami
Stan programu	Zmienny, modyfikowany przez instrukcje	Często brak pojęcia stanu	Zazwyczaj niemutowalny (brak efektów ubocznych)	Opisywany przez fakty (nie zmienia się w czasie)	Stan przechowywany wewnątrz obiektów
Podstawowy element	Instrukcja	Wyrażenie / deklaracja	Funkcja	Fakt i reguła	Klasa i obiekt
Reużywalność kodu	Procedury i funkcje	Szablony, zapytania	Funkcje wyższego rzędu	Reguły logiczne	Dziedziczenie, interfejsy
Model rzeczywistości	Ciąg działań	Opis celu	Transformacja danych	Związek między faktami	Struktura obiektów i ich relacje
Przykładowe języki	C, Pascal, Python (w stylu proceduralnym)	SQL, HTML, CSS, LINQ	F#, Haskell, LISP	Prolog	C#, Java, C++, Python (OOP)
Opis w jednym zdaniu	„Powiedz komputerowi, co ma robić krok po kroku.”	„Powiedz, jaki chcesz mieć wynik.”	„Traktuj obliczenia jak funkcje matematyczne.”	„Podaj fakty, a komputer wywnioskuje odpowiedź.”	„Modeluj świat jako współpracujące obiekty.”

Paradygmatów jest więcej...

Inne przykładowe paradygmaty, o których istnieniu warto wiedzieć:

- **Proceduralny** – program dzieli się na procedury i funkcje, które wykonują określone zadania w ustalonej kolejności.
- **Modularny** – kod organizuje się w niezależne moduły, które można rozwijać i testować osobno.
- **Event-driven** (zdarzeniowy) – program reaguje na zdarzenia (np. kliknięcia, sygnały, komunikaty) zamiast wykonywać instrukcje sekwencyjnie.
- **Sterowany danymi** (*data-driven*) – przepływ programu zależy od napływających danych, a nie od z góry ustalonego algorytmu.
- **Programowanie zorientowane na komponenty** – aplikacja składa się z gotowych, wymiennych komponentów o jasno określonych interfejsach.
- **Programowanie reaktywne** – koncentruje się na strumieniach danych i propagacji zmian w czasie rzeczywistym.
- **Programowanie zorientowane na usługi** (SOA) – system buduje się z usług komunikujących się przez sieć za pomocą ustalonych protokołów.
- **Programowanie równoległe / współbieżne** – skupia się na wykonywaniu wielu zadań jednocześnie dla efektywnego wykorzystania zasobów.
- **Programowanie symboliczne** – operuje na symbolach i wyrażeniach matematycznych zamiast na liczbach (np. w systemach AI).
- **Programowanie genetyczne** – wykorzystuje mechanizmy ewolucji biologicznej (mutacje, krzyżowanie, selekcję) do generowania rozwiązań.

Paradygmaty w C#

C# jest **wieloparadygmatowym** językiem programowania, co oznacza, że wspiera wiele różnych stylów programowania:

- **Imperatywny** – większość kodu C# to instrukcje, które wykonują się sekwencyjnie (if, for, while, przypisania itd.).
- **Obiektowy (OOP)** – podstawowy paradygmat C#, z klasami, dziedziczeniem, polimorfizmem, enkapsulacją i abstrakcją.
- **Proceduralny** – kod może być organizowany w metody (procedury), nawet bez tworzenia złożonych struktur obiektowych.
- **Funkcyjny** – C# od wersji 3.0 wspiera funkcje wyższego rzędu, lambdy, LINQ, niemutowalne typy i wyrażenia funkcyjne.
- **Deklaratywny** – w szczególności dzięki LINQ, które pozwala deklaratywnie określać, co chcemy uzyskać z danych, nie jak to zrobić.
- **Event-driven (zdarzeniowy)** – dzięki zdarzeniom (event, delegate), obsłudze interfejsów GUI i reakcji na sygnały systemowe.
- **Reaktywny** – możliwy z bibliotekami takimi jak Reactive Extensions (Rx.NET) lub IObservable<T>, które wspierają strumienie danych i subskrypcje.
- **Asynchroniczny / współbieżny** – poprzez async, await, Task, Parallel, Thread, Channel, IAsyncEnumerable itd.
- **Komponentowy** – aplikacje można budować jako zestawy współpracujących komponentów (np. w Blazorze, WPF, WinForms, MAUI).
- **Zorientowany na usługi (SOA / mikroserwisy)** – dzięki ASP.NET Core, gRPC, Web API – C# nadaje się do budowy systemów usługowych.

C# łączy imperatywną podstawę z obiektową strukturą i funkcyjnym sposobem myślenia. Daje też narzędzia do programowania deklaratywnego, zdarzeniowego i asynchronicznego, co czyni go jednym z najbardziej elastycznych języków współczesnych.

Metody

Procedury

Definicja:

Procedura to **fragment kodu**, który wykonuje określoną sekwencję instrukcji, **nie zwracając żadnej wartości**.

Funkcje

Definicja:

Funkcja to **fragment kodu**, który **wykonuje operacje i zwraca wartość**. Zawsze posiada **typ zwracany** (np. int, string, bool).

Metody

Definicja:

Metoda to **fragment kodu** (zestaw instrukcji) przypisany do **klasy**, **struktury** lub **interfejsu**. Wykonuje określone zadanie i **może** zwracać wartość.

Charakterystyka:

- Może przyjmować parametry (wejścia).
- Może zwracać wynik, ale nie musi.
- Należy zawsze do jakiegoś obiektu lub typu (nie istnieje samodzielnie).
- Jest wywoływana przez nazwę obiektu lub nazwę klasy (jeśli statyczna).

```
class Powitanie
{
    public void Wyszwietl(string imie)
    {
        Console.WriteLine($"Cześć, {imie}!");
    }
}

//Wywołanie:
var powitanie = new Powitanie();
powitanie.Wyszwietl("Jan");
```

*W C# każda funkcja i każda procedura to po prostu **metoda** – różni je tylko **typ zwracany** i czy **należą do instancji**, czy do **klasy**.
Każdy fragment kodu musi należeć do **jakiejś klasy lub struktury**, więc **wszystko jest metodą klasy**.*

Metody

Metoda to nie tylko nazwa i nawiasy – składa się z kilku ważnych elementów, które mają swoje konkretne znaczenie.

```
public int Dodaj(int a, int b)
{
    return a + b;
}
```

Sygnatura metody = nazwa metody + lista typów parametrów
(nie obejmuje typu zwracanego ani modyfikatorów dostępu)

`int Dodaj(int a, int b)`

Element	Nazwa	Opis
public	Modyfikator dostępu	Określa, kto może wywołać metodę (np. public, private, protected)
int	Typ zwracany	Określa, jaki typ danych metoda zwróci (tu: liczba całkowita)
Dodaj	Nazwa metody	Identyfikuje metodę w klasie – powinna być opisowa i zaczynać się wielką literą
(int a, int b)	Lista parametrów	Dane wejściowe przekazywane do metody
a,b	Parametry (argumenty formalne)	Zmiennice lokalne używane wewnątrz metody
return a + b;	Instrukcja zwracająca wartość	Zwraca wynik obliczeń zgodny z typem zwracanym

Metody void

Metoda:

```
class Logger
{
    //Przykład metody
    public void WypiszKomunikat(string tekst)
    {
        Console.WriteLine($"[INFO] {tekst}");
    }
}
```

Wywołanie:

```
var log = new Logger();
log.WypiszKomunikat("Aplikacja wystartowała");
```

Wynik w konsoli:

```
[INFO] Aplikacja wystartowała
```

Słowo kluczowe void oznacza, że **metoda nie zwraca żadnej wartości**. Używamy jej, gdy chcemy **coś wykonać**, ale **nie potrzebujemy wyniku**.

- Zamiast typu zwracanego (np. int, string), wpisujemy void.
- Nie mogą używać return z wartością — mogą tylko zakończyć działanie (return; bez niczego).
- Często służą do wyświetlania danych, zapisywania do pliku, zmiany stanu obiektu, itp.

Metody zwracające wartość

Metody zwracające wartość to takie, które **zwracają wynik działania** za pomocą słowa kluczowego **return**. Zawsze muszą mieć **określony typ zwracany** (np. `int`, `string`, `bool`, `double`, a nawet własny typ klasy).

- W nagłówku metody podajemy typ zwracany (zamiast `void`).
- Muszą kończyć się instrukcją `return`, która zwraca wartość tego typu.
- Wartość zwrócona może zostać przypisana do zmiennej.
- Możemy zwracać dowolny typ danych, również obiekty i kolekcje.

Metoda:

```
class Matematyka
{
    public int Pomnoz(int a, int b)
    {
        return a * b;
    }
}
```

Wywołanie:

```
var m = new Matematyka();
int wynik = m.Pomnoz(4, 5);
Console.WriteLine($"Wynik: {wynik}");
```

Metody zwracające wartość

W C# każda metoda z innym typem niż void (czyli zwracająca jakąś wartość), musi zwracać wartość we wszystkich swoich ścieżkach, inaczej program się nie skompiluje.

```
int Podziel(int a, int b)
{
    if (b != 0)
    {
        return a / b;
    }

    else if (a == 0)
    {
        Console.WriteLine("0 / 0 jest nieokreślone.");
        // brak return!
    }

    // brak return także tutaj!
}

//CS0161: 'Podziel(int, int)': not all code paths return a value
```

Źle 😞

```
int Podziel(int a, int b)
{
    if (b != 0)
    {
        return a / b;
    }

    else if (a == 0)
    {
        Console.WriteLine("0 / 0 jest nieokreślone.");
    }

    else
    {
        Console.WriteLine("Nie można dzielić przez zero!");
    }

    return 0;
}
```

Dobrze! 😍

Argumenty i parametry metod

Argumenty (lub **parametry**) to **wartości przekazywane do metody** w momencie jej wywołania. Dzięki nim metoda może **pracować na różnych danych**, bez potrzeby przepisywania kodu.

```
void Powitaj(string imie) // <-- Parametry znajdują się w definicji metody
{
    Console.WriteLine($"Cześć, {imie}!");
}

Powitaj("Jan"); // <-- Argumenty przekazujemy do metody w momencie jej
Powitaj("Kasia"); //      wywołania
```

Uwaga!

Liczba argumentów musi być równa liczbie parametrów.
Kolejność przekazywanych argumentów ma znaczenie.

Parametry domyślne

```
int Potega(int a, int b = 2)
{
    return (int)Math.Pow(a, b);
}

//Wywołanie:
Console.WriteLine(Potega(3)); // 9 -> używa wartości domyślnej b = 2
Console.WriteLine(Potega(3, 3)); // 27 -> nadpisuje wartość domyślną
```

Uwaga!

Parametr z wartością domyślną, nie może poprzedzać parametru obowiązkowego!

Parametr domyślny to taki, który ma z góry przypisaną **wartość**, jeśli **nie zostanie podany** podczas wywołania metody.

Zasady:

- Parametry domyślne **muszą** znajdować się na końcu listy parametrów.
- Wartość domyślna **musi być stała** w momencie kompilacji (np. liczba, string, null).
- Można ich używać również w metodach, konstruktorach, a nawet w interfejsach (od C# 8.0).

```
int Potega(int a = 2, int b) //Błąd kompilacji! ❌
```

Słowo kluczowe **default**

Słowo kluczowe **default** pozwala przypisać **domyślną wartość typu** – czyli **wartość początkową** dla danego typu danych (np. 0, false, null).

```
void WypiszLiczbe(int liczba = default)
{
    Console.WriteLine(liczba);
}

WypiszLiczbe();           // wypisze 0
WypiszLiczbe(10);        // wypisze 10
```

Słowo kluczowe default

Parametr domyślny **default** w metodzie generycznej

```
T PobierzWartosc<T>(bool warunek, T wartosc)
{
    if (warunek)
        return wartosc;

    else
        return default;
}
```

Z parametrów domyślnych z domyślną wartością typu korzysta się najczęściej przy metodach **przyjmujących parametry generyczne** oraz w **metodach asynchronicznych**.

Parametr domyślny **default** w metodzie asynchronicznej

```
async Task PobierzDaneAsync(CancellationTokencancellationTokencancellationToken = default)
{
    Console.WriteLine("Rozpoczynam pobieranie danych...");

    // Symulacja pracy asynchronicznej
    await Task.Delay(3000, cancellationTokencancellationToken);

    Console.WriteLine("Dane zostały pobrane!");
}
```

Przekazywanie argumentów

W C# istnieją dwa główne sposoby przekazywania argumentów do metody:

- przez wartość,
- przez referencję.

Przekazywanie argumentu przez wartość

Przekazywanie argumentów przez wartość to domyślny sposób przekazywania argumentów w C#. Metoda otrzymuje **kopię wartości** argumentu, więc wszelkie zmiany dotyczą tylko tej kopii – **oryginalna zmienna pozostaje bez zmian**.

```
void ZmienLiczbe(int x)
{
    x = 100;
}

int liczba = 5;
ZmienLiczbe(liczba);

Console.WriteLine(liczba); // wynik: 5
```

Metoda zmienia **kopię** wartości *Liczba*, ale oryginalna **pozostaje taka sama**.

Przekazywanie argumentu przez referencję

W przypadku **typów referencyjnych**, do metody przekazywana jest **referencja (adres)** obiektu w pamięci, a nie jego kopia. Metoda więc **może zmienić zawartość obiektu**.

```
void ZmienElementy(int[] liczby)
{
    liczby[0] = 100;
    liczby[1] = 200;
}

int[] dane = { 1, 2, 3 };
ZmienElementy(dane);

Console.WriteLine(string.Join(", ", dane));
// Wynik: 100, 200, 3
```

Metoda zmieniła elementy oryginalnej tablicy, bo zarówno dane, jak i liczby wskazują na ten sam obiekt w pamięci.

Przekazywanie argumentu przez referencję

```
void PodmienTablice(int[] liczby)
{
    liczby = new int[] { 9, 9, 9 }; //działa tylko lokalnie w metodzie
}

int[] dane = { 1, 2, 3 };
PodmienTablice(dane);

Console.WriteLine(string.Join(" ", dane));
//Wynik: 1, 2, 3 (oryginalna tablica nie została podmieniona)
```

Uwaga!

Metoda może zmienić zawartość tablicy, ale **nie może podmienić całej tablicy**.

Warto zapamiętać:

Co metoda może zrobić	Tak/Nie	Opis
Zmienić wartości elementów tablicy	✓ Tak	Bo odwołuje się do tego samego obiektu
Podmienić całą tablicę	✗ Nie	Zmienna w metodzie to tylko kopia referencji
Wskazać nowy obiekt na zewnątrz	✗ Nie (bez ref)	

Słowo kluczowe ref

Słowo kluczowe **ref** pozwala **przekazać zmienną przez referencję**, czyli metoda otrzymuje **bezpośredni dostęp do oryginalnej zmiennej**, a nie jej kopię. Dzięki temu zmiana wartości wewnątrz metody **wpływa na oryginał**.

Zasady:

- Zmienna przekazywana przez ref **musi być zainicjalizowana** przed wywołaniem metody.
- Zarówno przy **definicji**, jak i przy **wywołaniu** metody trzeba użyć słowa ref.
- ref działa zarówno z **typami wartościowymi**, jak i **referencyjnymi**.

```
void Zmien(ref int x)
{
    x = 100;
}

int liczba = 5;
Zmien(ref liczba);

Console.WriteLine(liczba); //wynik: 100
```

Dzięki **ref**, metoda operuje **na tym samym miejscu w pamięci**, więc oryginalna zmienna zostaje zmieniona.

Słowo kluczowe **ref** z typami referencyjnymi

Dla **typów referencyjnych** (*class*, *string*, *array* itd.) słowo kluczowe **ref** nie przekazuje „danych obiektu” (bo to i tak referencja), lecz pozwala **podmienić samą referencję** — czyli sprawić, że zmienna **będzie wskazywać na inny obiekt** po zakończeniu metody.

```
class Osoba
{
    public string Imie;
}

void Podmien(ref Osoba o)
{
    o = new Osoba { Imie = "Kasia" };
}

var osoba = new Osoba { Imie = "Jan" };
Podmien(ref osoba);

Console.WriteLine(osoba.Imie); // wynik: Kasia
```

Dzięki **ref** metoda **nie tylko modyfikuje obiekt**, ale też może **zmienić, na jaki obiekt zmienna wskazuje**.

Uwaga!

Bez słowa kluczowego **ref**, wynikiem byłby „Jan”.

Słowo kluczowe out

```
bool ParsujLiczbe(string tekst, out int wynik)
{
    if (Int32.TryParse(tekst, out wynik))
        return true;

    wynik = default;
    return false;
}

if (ParsujLiczbe("123", out int liczba))
{
    Console.WriteLine($"Sukces! Wynik: {liczba}");
}
else
{
    Console.WriteLine("Błąd parsowania.");
}
```

out pozwala metodzie zwrócić dodatkową wartość
– tu *liczba* *int*, oprócz wyniku *true/false*.

Słowo kluczowe **out** pozwala przekazać zmienną **przez referencję**, ale w przeciwieństwie do *ref* – **nie musi być zainicjalizowana przed wywołaniem metody**. Metoda ma obowiązek **nadać jej wartość** przed zakończeniem działania.

Zasady:

- Zmienna przekazywana jako *out* **nie musi być wcześniej zainicjalizowana**.
- Metoda **musi przypisać** wartość do zmiennej *out* zanim zakończy działanie.
- W definicji i wywołaniu metody **trzeba użyć** słowa *out*.
- *out* działa dla typów **wartościowych i referencyjnych**.

Słowo kluczowe out

W przypadku out, **kompilator wymaga**, żeby każda możliwa ścieżka w metodzie przypisała wartość do tej zmiennej.

```
bool SprawdzDane(string tekst, out int wynik)
{
    if (tekst == "OK")
    {
        wynik = 1;
        return true;
    }

    wynik = 0;
    return false;
}
```

Dobrze! 😍

```
bool SprawdzDane(string tekst, out int wynik)
{
    if (tekst == "OK")
    {
        wynik = 1;
        return true;
    }

    //Brak przypisania do 'wynik' w tej ścieżce!
    return false;
}

//Błąd kompilacji! ❌
//CS0177: The out parameter 'wynik' must be assigned to before control leaves the current method
```

Źle 😬

Porównanie ref i out

Oba słowa kluczowe pozwalają przekazać argument przez referencję, czyli metoda operuje na oryginalnej zmiennej, a nie na jej kopii. Różnią się jednak zachowaniem i przeznaczeniem:

Cecha	ref	out
Inicjalizacja przed wywołaniem	✅ Wymagana	❌ Niewymagana
Przypisanie wewnątrz metody	❌ Opcjonalne	✅ Wymagane (każda ścieżka musi przypisać wartość)
Cel użycia	Modyfikacja istniejącej wartości	Zwrócenie dodatkowych danych z metody
Liczba wartości zwracanych przez metodę	Zazwyczaj jedna	Może być wiele (out + wynik główny)
Typ danych	Może być wartość lub referencja	Może być wartość lub referencja
Użycie w deklaracji i wywołaniu	ref w obu miejscach	out w obu miejscach
Przykładowe zastosowanie	Zmiana wartości zmiennej przekazanej do metody	Parsowanie, walidacja, wielowartościowe wyniki (TryParse)

Przykład z życia:

ref – Daj mi kubek, żebym nalał więcej kawy.

out – Daj mi pusty kubek, ja go napelnię.

Słowo kluczowe **in**

Słowo kluczowe **in** przekazuje argument **przez referencję**, ale z gwarancją, że **metoda nie może zmienić jego wartości**. To tzw. **referencja tylko do odczytu** (*read-only reference*).

Zasady:

- Argument przekazywany przez **in** **nie może być zmieniony** wewnątrz metody.
- Wartość musi być **zainicjalizowana** przed przekazaniem.
- **in** działa z **typami wartościowymi i referencyjnymi**.
 - największy sens ma przy **dużych strukturach** (struct), by uniknąć kosztownego kopiowania.
- W wywołaniu **in** jest **opcjonalne** – można pominąć, jeśli kod jest czytelny.

```
void Wyszwietl(in int liczba)
{
    liczba = 10; // ❌ Błąd kompilacji – zmienna przekazana przez 'in' jest tylko do odczytu
    Console.WriteLine($"Wartość: {liczba}");
}

int x = 5;
Wyszwietl(in x);
```

Metoda może **odczytywać** wartość zmiennej, ale nie może jej **modyfikować**.

Słowo kluczowe params

```
void WswietlLiczby(params int[] liczby)
{
    foreach (int liczba in liczby)
    {
        Console.WriteLine(liczba);
    }
}

WswietlLiczby(1, 2, 3, 4, 5);
```

Metoda przyjmuje dowolną liczbę argumentów typu `int`, w środku widzi je jako **tablicę** (`int[]`).

Słowo **params** umożliwia przekazanie do metody **zmiennej liczby argumentów** tego samego typu. Metoda przyjmuje je jako **tablicę**, którą kompilator tworzy automatycznie.

Zasady:

- Parametr **params** musi być **ostatnim parametrem** w liście argumentów.
- W metodzie może być **tylko jeden** taki parametr.
- Można przekazać:
 - dowolną liczbę argumentów (0, 1, 5, 10...)
 - lub **gotową tablicę**.

Uwaga!

Parametr **params** działa **tylko dla tablic jednowymiarowych** i nie można po nim dodać innych parametrów.

Argumenty nazwane

Argumenty nazwane pozwalają określić **nazwę parametru** przy wywołaniu metody, zamiast przekazywać argumenty tylko **według kolejności**. Korzysta się z nich głównie, jeżeli dana metoda ma bardzo wiele parametrów (z czego wiele domyślnych), a my chcemy przekazać tylko ich część.

Zasady:

- Argumenty nazwane można łączyć z pozycyjnymi, **ale wszystkie nazwane muszą być po pozycyjnych**.

```
Test(5, nazwa: "abc"); // ✓  
Test(nazwa: "abc", 5); // ✗ błąd
```

- Nazwy muszą odpowiadać **nazwom parametrów w definicji metody**.
- Można ich używać **razem z parametrami domyślnymi**, aby przekazać tylko wybrane argumenty.

```
void Powitanie(string imie, int wiek)  
{  
    Console.WriteLine($"Cześć, {imie}! Masz {wiek} lat.");  
}  
  
// Wywołanie tradycyjne (po kolejności):  
Powitanie("Jan", 16);  
  
// Wywołanie z argumentami nazwanymi:  
Powitanie(wiek: 17, imie: "Kasia");
```

Oba wywołania są identycznie pod kątem działania.

Przeciążanie metod

```
void Wypisz(int liczba)
{
    Console.WriteLine($"Liczba: {liczba}");
}

void Wypisz(string tekst)
{
    Console.WriteLine($"Tekst: {tekst}");
}

void Wypisz(int a, int b)
{
    Console.WriteLine($"Suma: {a + b}");
}

Wypisz(5);
Wypisz("Hello");
Wypisz(3, 7);
```

Wszystkie metody mają **tę samą nazwę**, ale różnią się **sygnaturą** (czyli listą typów parametrów).

Przeciążanie metod polega na tworzeniu **wiele metod o tej samej nazwie**, ale z **różnymi parametrami** (*liczbą, typem lub kolejnością*). Dzięki temu jedna nazwa może wykonywać **podobne zadanie dla różnego zestawu danych**.

Unikalną sygnaturę metody tworzy:

- nazwa metody,
- typy i liczba parametrów,
- kolejność parametrów.

Uwaga!

Typ zwracany nie wpływa na przeciążenie!

```
int Oblicz() { ... }
double Oblicz() { ... } // ❌ błąd – taka sama sygnatura
```

Przeciążanie metod

Kiedy metoda jest przeciążona, a jedna z wersji ma **parametry domyślne**, kompilator może nie być w stanie rozstrzygnąć, którą wersję wywołać.

Kompilator nie wie, którą metodę wybrać:

- pierwszą Powitaj(string),
- czy drugą Powitaj(string, string = "Witaj").

Obie metody **pasują do wywołania z jednym argumentem**.

```
void Powitaj(string imie)
{
    Console.WriteLine($"Cześć, {imie}!");
}

void Powitaj(string imie, string przywitanie = "Witaj")
{
    Console.WriteLine($"{przywitanie}, {imie}!");
}

Powitaj("Jan");

//Błąd kompilacji! ✖
//CS0121: The call is ambiguous between the following methods...
```

Wniosek

Nie łącz przeciążania z parametrami domyślnymi w sytuacjach, w których wywołania mogłyby być **niejednoznaczne**.

Klasy

Czym jest klasa?

Klasa to **szablon (plan, wzorzec)**, na podstawie którego tworzy się **obiekty**.

Można ją porównać do projektu domu – sama klasa nie jest domem, ale określa, jak dom ma wyglądać: ile ma pokoi, jakie ma drzwi, jakie funkcje pełni kuchnia itd.

W świecie programowania:

- **Klasa** opisuje, jakie dane (czyli **poła** i **właściwości**) obiekt będzie przechowywał oraz jakie działania (**metody**) można na nim wykonać.
- **Obiekt** to konkretny egzemplarz klasy (*instancja*), np. *konkretny dom* według projektu.

Czym jest klasa?

```
// Definicja klasy
public class Samochod
{
    // Pole – przechowuje dane
    public string Marka;
    public string Kolor;
    public int Przebieg;

    // Metoda – opisuje zachowanie obiektu
    public void Jedz()
    {
        Console.WriteLine($"{Marka} jedzie!");
    }
}

// Użycie klasy (utworzenie obiektu)
Samochod auto = new Samochod();
auto.Marka = "BMW";
auto.Kolor = "Czarny";
auto.Przebieg = 120000;
auto.Jedz(); // Wyświetli: BMW jedzie!
```

Wyobraź sobie, że masz klasę Samochód. Różni kierowcy mogą mieć różne samochody – ale każdy samochód ma:

- markę,
- kolor,
- przebieg,
- metodę Jedź().

Czym jest klasa?

Zapamiętaj:

- Klasa to definicja obiektu, a nie sam obiekt.
- Obiekt to instancja klasy.
- W C# wszystkie klasy są typami referencyjnymi – ich dane trafiają na stertę (heap).
- Klasa może zawierać:
 - pola,
 - właściwości,
 - metody,
 - konstruktory,
 - zdarzenia,
 - klasy wewnętrzne,
 - i wiele innych elementów.

Definiowanie klasy

Klasa w C# definiowana jest za pomocą słowa kluczowego **class**, po którym następuje jej **nazwa**. Najprostsza możliwa definicja wygląda tak:

```
class Osoba  
{  
  
}
```

*To już **pełnoprawna klasa**, choć jeszcze nic nie robi.*

Składnia klasy



```
[<modyfikator_dostępu>] class <NazwaKlasy>
{
    // Pola (fields)
    // Właściwości (properties)
    // Metody (methods)
    // Konstruktory (constructors)
    // itd.
}
```

Definiowanie klasy

```
public class Osoba
{
    // Prywatne pole (backing field) – nazewnictwo: _wiek
    private int _wiek;

    // Publiczna właściwość – PascalCase
    public string Imie { get; set; } = string.Empty;

    // Właściwość z walidacją przez prywatne pole
    public int Wiek
    {
        get => _wiek;
        set
        {
            if (value < 0) throw new ArgumentOutOfRangeException(nameof(value), "Wiek nie może być ujemny.");
            _wiek = value;
        }
    }

    // Właściwość tylko do odczytu (np. wyliczana)
    public bool Pelnoletni => Wiek >= 18;

    // Właściwość z init-only (ustawiana tylko przy tworzeniu obiektu)
    public string Pesel { get; init; } = string.Empty;

    // Metoda – zachowanie obiektu
    public void PrzywitajSie()
    {
        Console.WriteLine($"Cześć, mam na imię {Imie} i mam {Wiek} lat.");
    }
}
```

Przykład prostej klasy

```
// Użycie:
var osoba = new Osoba
{
    Imie = "Kasia",
    Wiek = 25,
    Pesel = "01234567890" // init działa tylko tutaj (podczas inicjalizacji)
};
osoba.PrzywitajSie();
Console.WriteLine(osoba.Pelnoletni); // True
```

Tworzenie instancji klasy

Definiowanie klasy

Inaczej mówiąc:

- **Definicja klasy** = projekt, plan, przepis
 - mówimy, **czym jest ten typ** (jakie ma pola, właściwości, metody).
- **Tworzenie obiektu klasy (instancji)** = użycie tego planu
 - mówimy, **stwórz mi konkretny egzemplarz tego typu**.

Wniosek

Tworzenie klasy → definiujemy nowy typ (plan).

Tworzenie obiektu → tworzymy konkretny egzemplarz (instancję).

```
// Definicja klasy (czyli tworzenie klasy)
public class Pies
{
    public string Imie { get; set; }

    public void Szczekaj()
    {
        Console.WriteLine($"{Imie} szczeka!");
    }
}

// Utworzenie obiektu (instancji klasy)
Pies mojPies = new Pies();
mojPies.Imie = "Kiara";
mojPies.Szczekaj(); // "Kiara szczeka!"
```

Tworzenie instancji klasy

Modyfikatory dostępu

Modyfikator	Dostępność
public	Dostępny wszędzie , także poza klasą i projektem
private	Dostępny tylko wewnątrz tej klasy
protected	Dostępny w tej klasie i klasach pochodnych
internal	Dostępny w obrębie tego samego projektu (<i>assembly</i>)
protected internal	Dostępny dla klas pochodnych lub w tym samym projekcie
private protected	Dostępny dla klas pochodnych, ale tylko w tym samym projekcie

Tabela przedstawiająca modyfikatory dostępu

Modyfikatory dostępu

```
var konto = new KontoBankowe();
konto.Wlasciciel = "Jan Kowalski";
konto.Wplac(200);
// konto._saldo = 100; ❌ – pole prywatne, niedostępne
```

Przykład działania modyfikatorów dostępu

```
public class KontoBankowe
{
    // Prywatne pole – widoczne tylko w tej klasie
    private decimal _saldo;

    // Publiczna właściwość – widoczna wszędzie
    public string Wlasciciel { get; set; }

    // Protected metoda – dostępna tylko w klasach pochodnych
    protected void ZmienSaldo(decimal kwota)
    {
        _saldo += kwota;
    }

    // Publiczna metoda – interfejs dla użytkownika
    public void Wplac(decimal kwota)
    {
        ZmienSaldo(kwota);
    }

    // Internal metoda – dostępna w obrębie projektu
    internal void WyzerujSaldo()
    {
        _saldo = 0;
    }
}
```

Przykład klasy z modyfikatorami dostępu

Modyfikatory dostępu

W C# każdy element ma **domyślny modyfikator dostępu**, który zależy od **miejsca, w którym jest zadeklarowany**.

```
class Samochod
{
    string marka;
    void Uruchom() { }
}
```

Kontekst	Domyślny modyfikator
Klasa / struct (na poziomie pliku)	internal
Członek klasy / struct (pole, metoda, właściwość)	private
Członek interfejsu	public

Brak jawnego modyfikatora **nie oznacza „publicznego”** dostępu — wręcz przeciwnie – C# z zasady domyślnie **zamyka dostęp** (bezpieczeństwo przez domyślność).

Pola i właściwości

W klasach przechowujemy dane za pomocą **pól** i **właściwości**. Na pierwszy rzut oka wyglądają podobnie, ale mają **różne zastosowanie**.

Pole (field)

```
public class Samochod
{
    private int _predkosc; // pole prywatne

    public void UstawPredkosc(int kmh)
    {
        _predkosc = kmh;
    }

    public void PokazPredkosc()
    {
        Console.WriteLine($"Aktualna prędkość: {_predkosc} km/h");
    }
}
```

To **zmienna zadeklarowana bezpośrednio w klasie**, która przechowuje dane. Zazwyczaj jest **prywatna** (private) i służy jako **magazyn danych** dla właściwości.

Właściwość (property)

```
public class Samochod
{
    private int _predkosc;

    public int Predkosc
    {
        get { return _predkosc; }
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(value), "Prędkość nie może być ujemna.");

            _predkosc = value;
        }
    }
}
```

To **bezpieczny sposób udostępnienia danych** na zewnątrz klasy. Pozwala kontrolować odczyt i zapis poprzez **get** i **set**.

Zwróć uwagę na słowo kluczowe **value**

Właściwości automatyczne

Jeżeli nie musimy dodawać logiki, można pominąć pole `_predkosc` – C# sam tworzy je w tle.

```
public class Samochod
{
    public string Marka { get; set; }
    public int RokProdukcji { get; set; }
}
```

*Powyższe właściwości może zmieniać i odczytywać każdy – są one **publiczne do odczytu i zapisu**.*

Uwaga!

Nie powinniśmy nigdy tworzyć pól publicznych, do tego służą właściwości.
Publiczne pole byłoby niezgodne z zasadą enkapsulacji w OOP (o tym szerzej za tydzień).

Właściwości automatyczne

Właściwość tylko do odczytu
(ustawiana tylko wewnątrz klasy – w konstruktorze lub przy deklaracji)

```
public string NumerVin { get; } = "ABC123456";
```

Właściwość do odczytu i prywatnego zapisu

```
public string Haslo { private get; set; }
```

Właściwość do zapisu tylko podczas tworzenia obiektu

```
public class Uzytkownik
{
    public string Imie { get; init; }
    public string Nazwisko { get; init; }
}

var user = new Uzytkownik
{
    Imie = "Jan",
    Nazwisko = "Kowalski"
};

// user.Imie = "Adam"; ❌ – nie można zmienić po utworzeniu
```

Typ	Dostępność	Użycie
pole (<i>prywatne</i>)	tylko wewnątrz klasy	przechowywanie danych
public property	na zewnątrz klasy	kontrolowany dostęp do danych
get / set	odczyt / zapis	można dodać logikę (<i>walidację, przeliczenia</i>)
init	tylko przy tworzeniu obiektu	bezpieczeństwo i niemutowalność

Właściwości automatyczne

```
public class Samochod
{
    public string Marka { get; } = "BMW"; // tylko do odczytu
    public string Kolor { get; init; }    // można ustawić przy tworzeniu
}

var auto = new Samochod
{
    Kolor = "Czarny" // ✓ działa
};

// auto.Marka = "Audi"; ✗ nie działa
// auto.Kolor = "Biały"; ✗ też nie działa
```

Przykład korzystania z właściwości

Pola tylko do odczytu – readonly

readonly to modyfikator, który oznacza, że **wartość pola można przypisać tylko raz** – albo w momencie deklaracji, albo w konstruktorze.

Po tym etapie (czyli po utworzeniu obiektu) wartość **nie może już być zmieniona**.

```
public class Serwer
{
    private readonly string _adres;

    public Serwer(string adres)
    {
        _adres = adres; // ✅ dozwolone – w konstruktorze
    }

    public void WyszwietlAdres()
    {
        Console.WriteLine($"Serwer działa pod adresem: {_adres}");
    }

    public void ZmienAdres(string nowy)
    {
        // _adres = nowy; ❌ Błąd: nie można zmienić readonly po utworzeniu
    }
}
```

_adres można ustawić tylko **raz** – przy tworzeniu obiektu.
To sprawia, że obiekt staje się **bardziej przewidywalny i bezpieczny**.

Kiedy warto stosować pola `readonly`?

1. Gdy wartość się nie zmienia po inicjalizacji – np. unikalny identyfikator, adres serwera, konfiguracja aplikacji, katalog roboczy.

```
public class Konfiguracja
{
    private readonly string _plikKonfiguracyjny = "config.json";
    private readonly Guid _id = Guid.NewGuid();
}
```

Klasa konfiguracyjna – dane powinniśmy ustawiać tylko raz, najczęściej przy uruchamianiu programu.

Kiedy warto stosować pola `readonly`?

2. Gdy obiekt zależy od wartości przekazanych w konstruktorze. To bardzo częsty przypadek przy wstrzykiwaniu zależności (*dependency injection*).

```
public class UzytkownikService
{
    private readonly ILogger _logger;
    private readonly IRepository _repo;

    public UzytkownikService(ILogger logger, IRepository repo)
    {
        _logger = logger;
        _repo = repo;
    }

    public void Zapisz()
    {
        _logger.Log("Zapisano użytkownika.");
        _repo.Save();
    }
}
```

Dzięki `readonly` mamy gwarancję, że `_logger` i `_repo` **nie zostaną podmienione** po stworzeniu obiektu.
To kluczowe dla stabilności systemów wielowątkowych i serwisów.

Kiedy warto stosować pola `readonly`?

3. Gdy chcesz zachować stałość w obiektach niemutowalnych.

```
public class Punkt
{
    public readonly int X;
    public readonly int Y;

    public Punkt(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Wartości współrzędnych nie mogą się zmienić po utworzeniu punktu –
to tworzy **niemutowalny obiekt**, podobnie jak *record*.

Różnica między `const` i `readonly`

```
public class Serwer
{
    public const int DomyslnyPort = 80;
    public readonly int Port;

    public Serwer(int port)
    {
        Port = port;
    }
}
```

Przykład ilustrujący różnicę między `const` i `readonly`

Cecha	<code>const</code>	<code>readonly</code>
Przypisana w kompilacji	✓ tak	✗ nie
Przypisana w konstruktorze	✗ nie	✓ tak
Może zależeć od danych wejściowych	✗ nie	✓ tak
Typy dozwolone	tylko proste (int, string, bool itd.)	dowolne, także klasy
Modyfikacja po utworzeniu	✗ nie	✗ nie

Tworzenie instancji – słowo kluczowe **new**

Każda klasa w C# to **typ referencyjny**. Aby z niej korzystać, trzeba utworzyć **instancję (obiekt)** – robimy to przy pomocy słowa kluczowego **new**.

```
public class Samochod
{
    public string Marka { get; set; }
    public int Rok { get; set; }

    public void Wyszwietl()
    {
        Console.WriteLine($"{Marka} ({Rok})");
    }
}

// Tworzenie obiektu
var auto = new Samochod();
auto.Marka = "BMW";
auto.Rok = 2020;
auto.Wyszwietl(); // BMW (2020)
```

***new Samochod()** tworzy nowy obiekt, a zmienna **auto** przechowuje referencję do tego obiektu.*

Tworzenie instancji – słowo kluczowe **new**

Jeżeli chcemy, możemy od razu po utworzeniu obiektu ustawić jego właściwości.

```
var auto = new Samochod
{
    Marka = "Audi",
    Rok = 2021
};
```

Ten sposób działa, gdy klasa ma publiczne settery lub init.

Tworzenie instancji – słowo kluczowe **new**

Każda instancja klasy jest **typem referencyjnym**.



```
var auto1 = new Samochod { Marka = "Ford" };  
var auto2 = auto1;  
  
auto2.Marka = "Fiat";  
Console.WriteLine(auto1.Marka); // Fiat !
```

Obie zmienne wskazują **ten sam obiekt w pamięci**. Jeśli chcesz utworzyć **nowy, niezależny obiekt**, musisz użyć **new**.

Tworzenie instancji – słowo kluczowe **new**

Jeżeli klasa ma konstruktor przyjmujący parametry, możesz przekazać je bezpośrednio przy **new**.

```
public class Punkt
{
    public int X { get; }
    public int Y { get; }

    public Punkt(int x, int y)
    {
        X = x;
        Y = y;
    }
}

var p = new Punkt(3, 5);
Console.WriteLine($"{p.X}, {p.Y}"); // (3, 5)
```

Tworzenie instancji – słowo kluczowe **new**

Jeśli typ jest znany z kontekstu, nie trzeba pisać go dwa razy



```
Samochod auto = new(); // równoważne z new Samochod()  
auto.Marka = "Volvo";
```

Uwaga!

Ten sposób oczywiście nie zadziała z `var`.

Tworzenie instancji – słowo kluczowe new

```
1 odwołanie | 0 zmian | 0 autorów, 0 zmian
public MarketInfo GetMarketInfo(Guid botId, long eventId)
{
    if (_bets.TryGetValue(key: botId, value: out var List<MatchedBetModel>? bets))
    {
        //Assuming that all bets for the same event have the same market id
        if (bets.Count > 0)
        {
            var MatchedBetModel? bet = bets.FirstOrDefault(MatchedBetModel b => b.EventId == eventId);
            if (bet != null)
            {
                return new MarketInfo() ←
                {
                    EventId = bet.EventId,
                    MarketId = bet.MarketId,
                    MarketName = bet.MarketName,
                    Country = (Countries)bet.CountryId
                };
            }
        }
    }

    return null;
}
```

*Prawdziwy przykład korzystania z **new()**, jeżeli znany jest kontekst.
Kompilator automatycznie wnioskuję że skoro przed **new** jest **return**, typem tworzonym jest typ zwracany.*

Konstruktory

Konstruktor to specjalna metoda, która uruchamia się **automatycznie** w momencie tworzenia obiektu (**new**). Służy do inicjalizacji pól lub właściwości – czyli ustawienia stanu początkowego obiektu.

```
public class Osoba
{
    private string _imie;

    // Konstruktor
    public Osoba(string imie)
    {
        _imie = imie;
        Console.WriteLine("Tworzę obiekt klasy Osoba...");
    }
}
```

Definicja konstruktora

```
var o = new Osoba("Kasia"); // automatycznie wywołuje konstruktor
```

Wywołanie konstruktora

Konstruktor domyślny

Jeśli nie napiszesz żadnego konstruktora, kompilator automatycznie utworzy **pusty konstruktor bez parametrów**.



```
public class Zwierze
{
    public string Gatunek { get; set; }
}

// działa mimo braku konstruktora
var z = new Zwierze();
```

Uwaga!

Ale jeśli zdefiniujesz jakikolwiek konstruktor, kompilator nie doda już domyślnego.

Konstruktor z parametrami

```
public class Samochod
{
    public string Marka { get; }
    public int Rok { get; }

    public Samochod(string marka, int rok)
    {
        Marka = marka;
        Rok = rok;
    }
}

var auto = new Samochod("BMW", 2020);
```

Konstruktor może przyjmować argumenty (być sparametryzowany).

Konstruktor przeciążony

W jednej klasie możesz mieć **kilka konstruktorów** z różnymi zestawami parametrów.

```
public class Uzytkownik
{
    public string Imie { get; }
    public int Wiek { get; }

    public Uzytkownik(string imie)
    {
        Imie = imie;
        Wiek = 0;
    }

    public Uzytkownik(string imie, int wiek)
    {
        Imie = imie;
        Wiek = wiek;
    }
}

var u1 = new Uzytkownik("Adam");
var u2 = new Uzytkownik("Ewa", 25);
```

C# wybierze właściwy konstruktor, na podstawie przekazanych argumentów.

Wywołanie innego konstruktora

Czasem chcesz, żeby jeden konstruktor wywoływał drugi, żeby nie duplikować kodu inicjalizacji.

```
public class Punkt
{
    public int X { get; }
    public int Y { get; }

    public Punkt() : this(0, 0) // wywołuje inny konstruktor
    {
    }

    public Punkt(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Ten konstruktor wykona się pierwszy.

Wywołanie drugiego konstruktora różni się, od wywołania metody.

Prywatny konstruktor

Konstruktor poprzedzony słowem kluczowym **private** służy do **ograniczenia tworzenia obiektów**. Taki zabieg stosuje się przykładowo w przypadku wzorca projektowego **Singleton**.

```
public class Logger
{
    private static readonly Logger _instancja = new Logger();

    private Logger() { } // nikt spoza klasy nie może stworzyć obiektu

    public static Logger Instancja => _instancja;
}
```

```
//Jeżeli spróbujemy utworzyć obiekt z zewnątrz klasy, program się nie skompiluje.
var logger = new Logger(); // ❌ Błąd kompilacji!
```

Próba stworzenia obiektu z zewnątrz.

Słowo kluczowe **this**

Słowo kluczowe **this** oznacza po prostu **aktualny obiekt**, na którym wywoływana jest metoda lub właściwość. Najczęściej stosuje się przy kopiowaniu danych między klasami reprezentującymi ten sam obiekt.

```
public class StudentModel
{
    public string Name { get; set; }
}

public class Student
{
    public string Name { get; set; }

    public StudentModel ToModel()
    {
        return new StudentModel
        {
            // this wskazuje na bieżącego studenta
            Name = this.Name
        };
    }
}
```

Klasa Student może być klasą bazowaną, StudentModel to klasa modelu.

```
return new StudentModel
{
    Name = Name // działa, ale nie widać jasno skąd pochodzi wartość
};
```

Nieczytelne, może prowadzić do błędów.

Klasy wewnętrzne

W C# możesz **zagnieżdżać** jedną klasę w drugiej, czyli umieszczać klasę **wewnątrz innej klasy**. Taką klasę nazywamy **klasą wewnętrzną** (*nested class*).

```
public class Samochod
{
    public string Marka { get; set; }

    // Klasa wewnętrzna
    public class Silnik
    {
        public int Moc { get; set; }
    }
}
```

```
var silnik = new Samochod.Silnik();
silnik.Moc = 200;

var auto = new Samochod { Marka = "BMW" };
Console.WriteLine($"{auto.Marka} ma silnik o mocy {silnik.Moc} KM");
```

Klasa `Silnik` istnieje **tylko w kontekście klasy `Samochod`**. Z punktu widzenia nazewnictwa jest to `Samochod.Silnik`.

Klasy wewnętrzne

Klasy wewnętrznych najczęściej używa do **ukrywania implementacji**. Klasa wewnętrzna zwykle jest **klasą tylko pomocniczą**, i nie jest widoczna na zewnątrz.

```
public class Logger
{
    // Klasa pomocnicza – prywatna
    private class LogEntry
    {
        public DateTime Timestamp { get; set; } = DateTime.Now;
        public string Message { get; set; }
    }

    public void Zapisz(string msg)
    {
        var entry = new LogEntry { Message = msg };
        Console.WriteLine($"[{entry.Timestamp}] {entry.Message}");
    }
}
```

Klasa LogEntry jest ukryta wewnątrz Logger i nie można jej użyć poza nim. To świetny sposób na hermetyzację.

Klasy statyczne – słowo kluczowe static

```
public static class Kalkulator
{
    public static int Dodaj(int a, int b)
    {
        return a + b;
    }

    public static int Pomnoz(int a, int b)
    {
        return a * b;
    }
}

// Użycie:
int wynik = Kalkulator.Dodaj(2, 3);
Console.WriteLine(wynik); // 5
```

Klasa statyczna to klasa, której:

- nie można utworzyć za pomocą new,
- zawiera wyłącznie **składowe statyczne** (pola, właściwości, metody).

Używana jest jako **kontener dla funkcji lub danych wspólnych** dla całej aplikacji, a nie dla konkretnych obiektów.

Nie trzeba tworzyć obiektu `new Kalkulator()` – odwołujemy się **bezpośrednio** przez nazwę klasy.

Klasy statyczne

Cechy klas statycznych:

- zawsze posiadają **static** przed nazwą – blokuje możliwość tworzenia jej instancji,
- zawierają tylko elementy statyczne – wszystkie pola, metody i właściwości też muszą być statyczne,
- nie można dziedziczyć – ani z klasy statycznej, ani jako klasa statyczna (*jako pochodna*).

Cecha	Klasa zwykła	Klasa statyczna
Tworzenie instancji	✅ tak (new)	❌ nie
Pola i metody	✅ instancyjne i statyczne	⚠️ tylko statyczne
Dziedziczenie	✅ możliwe	❌ nie
Interfejsy	✅ można implementować	❌ nie można
Konstruktor	✅ dowolny	⚠️ tylko statyczny

Różnice między klasami zwykłymi a statycznymi.

Metody i właściwości statyczne

Pola i właściwości statyczne należą **do klasy**, nie do konkretnego obiektu.

```
//Klasa nie musi być statyczna, żeby posiadać statyczne metody i właściwości
public class Serwer
{
    public static int LiczbaPolaczen { get; private set; }

    public static void Polacz()
    {
        LiczbaPolaczen++;
        Console.WriteLine($"Połączeń: {LiczbaPolaczen}");
    }
}
```

```
Serwer.Polacz();
Serwer.Polacz();
Console.WriteLine(Serwer.LiczbaPolaczen); // 2
```

LiczbaPolaczen jest wspólna dla całej klasy – niezależnie, ile razy wywołasz metodę.

Pole statyczne a instancyjne

```
public class Licznik
{
    // Pole statyczne – wspólne dla wszystkich obiektów
    private static int _globalCount = 0;

    // Pole instancyjne – niezależne dla każdego obiektu
    private int _localCount = 0;

    // Metoda instancyjna – operuje na danych konkretnego obiektu
    public void Zwiększ()
    {
        _globalCount++; // współdzielone między wszystkimi obiektami
        _localCount++; // tylko dla tego konkretnego obiektu
    }

    // Metoda instancyjna – wyświetla stan bieżącego obiektu
    public void Pokaz()
    {
        Console.WriteLine($"GlobalCount = {_globalCount}, LocalCount = {_localCount}");
    }

    // Metoda statyczna – pokazuje stan współdzielony (bez potrzeby tworzenia obiektu)
    public static void PokazGlobalnyStan()
    {
        Console.WriteLine($"GlobalCount = {_globalCount}");
    }
}
```

```
var a = new Licznik();
var b = new Licznik();

a.Zwiększ(); // zwiększa oba liczniki (globalny i lokalny a)
a.Pokaz();   // GlobalCount = 1, LocalCount = 1

b.Zwiększ(); // zwiększa oba liczniki (globalny i lokalny b)
b.Pokaz();   // GlobalCount = 2, LocalCount = 1

a.Pokaz();   // GlobalCount = 2, LocalCount = 1 (a ma własny lokalny licznik)

Licznik.PokazGlobalnyStan(); // GlobalCount = 2
```

Wnioski:

- Pola statyczne istnieją tylko w jednej kopii w całym programie
 - są współdzielone między wszystkimi instancjami klasy.
- Pola instancyjne tworzą się osobno dla każdego obiektu.
- Metody instancyjne mogą modyfikować zarówno pola instancyjne, jak i statyczne.
- Metody statyczne mogą modyfikować tylko pola statyczne.

GlobalCount jest wspólne dla całej klasy – niezależnie od liczby obiektów.
LocalCount należy do każdego obiektu osobno.

Konstruktor statyczny

Konstruktor statyczny w C# służy do **inicjalizacji danych statycznych klasy** lub do **wykonania jednorazowej akcji**. Jest on **wywoływany automatycznie** przez środowisko uruchomieniowe **tylko raz**, **przed utworzeniem pierwszej instancji danej klasy** lub **przed odwołaniem się do jakiegokolwiek jej statycznego składnika**.

Kluczowe cechy konstruktora statycznego

- **Automatyczne wywołanie:** Nie można go wywołać bezpośrednio w kodzie. Środowisko uruchomieniowe .NET zarządza jego wykonaniem.
- **Jednorazowe wykonanie:** Konstruktor statyczny dla danej klasy jest wykonywany co najwyżej jeden raz w trakcie działania programu.
- **Brak parametrów:** Konstruktor statyczny nie może przyjmować żadnych parametrów.
- **Brak modyfikatorów dostępu:** Nie może mieć modyfikatorów dostępu, takich jak `public` czy `private`. Jest on wywoływany przez system, a nie przez użytkownika.

Cel: Jego głównym zadaniem jest inicjalizacja pól statycznych lub wykonanie operacji, która jest niezbędna dla całej klasy, a nie dla jej poszczególnych obiektów.

Konstruktor statyczny

```
public static class Ustawienia
{
    public static string Wersja { get; }

    static Ustawienia()
    {
        Console.WriteLine("Inicjalizacja ustawień...");
        Wersja = "1.0.0";
    }
}
```

Przykład przedstawiający statyczny konstruktor.

Klasy cząstkowe – słowo kluczowe **partial**

Słowo kluczowe **partial** pozwala **podzielić definicję jednej klasy na kilka plików**. Wszystkie części klasy są **scalane w jedną** podczas kompilacji.

Dzięki temu możesz:

- organizować duże klasy (np. rozdzielić logikę, właściwości, metody),
- oddzielać kod ręcznie pisany od kodu generowanego automatycznie (np. przez Visual Studio, Entity Framework, Blazora).

User.Part1.cs

```
public partial class User
{
    public string Name { get; set; }

    public void Greet()
    {
        Console.WriteLine($"Hello, {Name}!");
    }
}
```

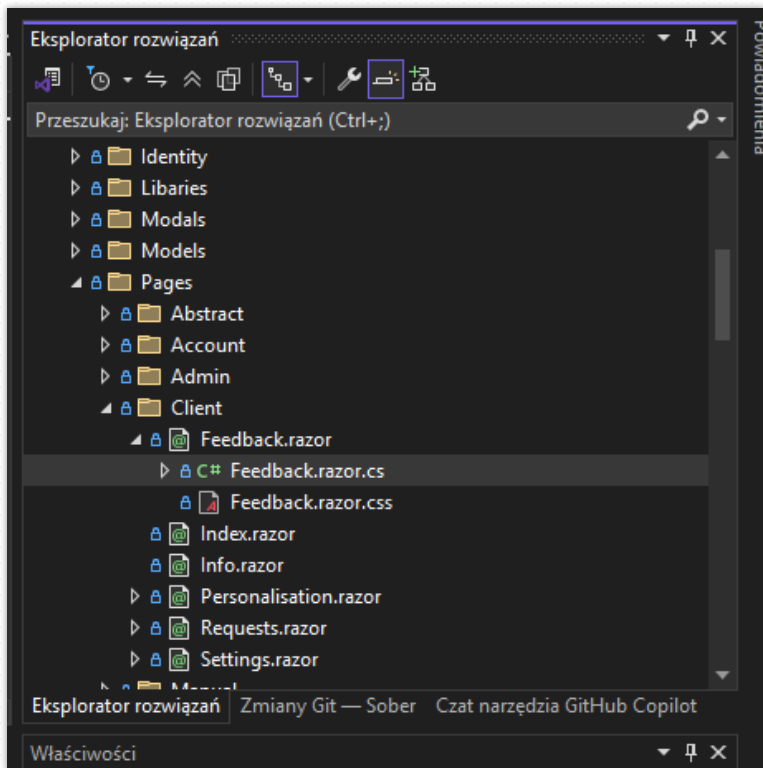
User.Part2.cs

```
public partial class User
{
    public string Role { get; set; }

    public void ShowRole()
    {
        Console.WriteLine($"{Name} is a {Role}");
    }
}
```

```
var user = new User { Name = "Anna", Role = "Admin" };
user.Greet(); // Hello, Anna!
user.ShowRole(); // Anna is a Admin
```

Obie części tworzą jedną klasę **User**. Podczas kompilacji są traktowane, jak jedna definicja klasy.

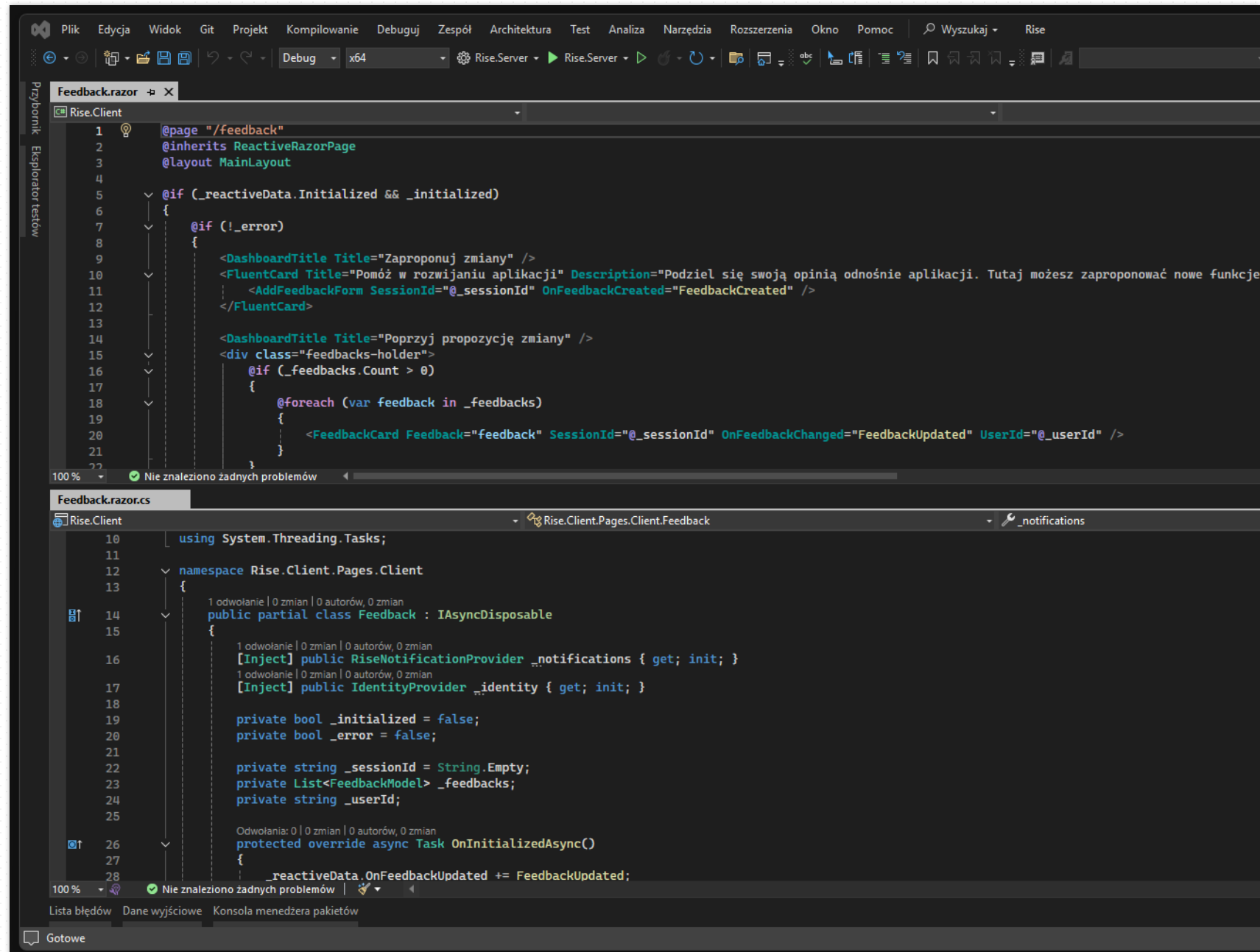


Tutaj praktyczny przykład wykorzystania klas częściowych w frameworku **Blazor**.

Klasą jest zarówno strona razor, jak i jej „code-behind” w osobnym pliku.

Uwaga!

Poszczególne części klasy częściowej, muszą znajdować się w jednym projekcie.



Metody rozszerzeń

Metoda rozszerzająca (extension method) to specjalna metoda, która pozwala **dodać nową funkcjonalność do istniejącego typu**, bez potrzeby dziedziczenia lub modyfikacji jego kodu źródłowego.

Zasady:

- musi znajdować się w **statycznej klasie**,
- musi być **statyczna**,
- jej **pierwszy parametr** musi mieć przed sobą słowo kluczowe **this** – i wskazuje, który typ rozszerzamy.

```
public static class StringExtensions
{
    public static string ToTitleCase(this string text)
    {
        if (string.IsNullOrEmpty(text))
            return text;

        return char.ToUpper(text[0]) + text.Substring(1).ToLower();
    }
}
```



```
string name = "kaSIA";
Console.WriteLine(name.ToTitleCase()); // Kasia
```



```
//Kompilator zmienia wywołanie z:
name.ToTitleCase();

//Do:
StringExtensions.ToTitleCase(name);
```

Przestrzenie nazw

```
namespace MyApp.Models
{
    public class User
    {
        public string Name { get; set; }
    }
}

namespace MyApp.Services
{
    public class UserService
    {
        public void Save(MyApp.Models.User user)
        {
            Console.WriteLine($"Zapisano użytkownika: {user.Name}");
        }
    }
}
```

Przestrzeń nazw (namespace) to sposób **grupowania klas, struktur, interfejsów i innych typów** w logiczne jednostki.

Dzięki nim:

- unikasz konfliktów nazw między różnymi klasami,
- łatwiej organizujesz projekt,
- możesz tworzyć czytelną hierarchię (np. MyApp.Services.Logging).

Dzięki przestrzeniom nazw możesz mieć wiele klas o tej samej nazwie w różnych miejscach projektu.

Importowanie przestrzeni nazw - **using**

Aby móc korzystać z klas z innej przestrzeni, używamy słowa kluczowego **using**.



```
using MyApp.Models;  
  
var user = new User { Name = "Kasia" };
```

Dzięki **using** nie trzeba pisać pełnej ścieżki `MyApp.Models.User`

Zagnieżdżone przestrzenie nazw

Możemy tworzyć hierarchię przestrzeni nazw – zgodnie z katalogami w projekcie.

```
namespace MyApp.Core.Services.Logging
{
    public class Logger { }
}
```

Użycie – wariant A

```
var log = new MyApp.Core.Services.Logging.Logger();
```

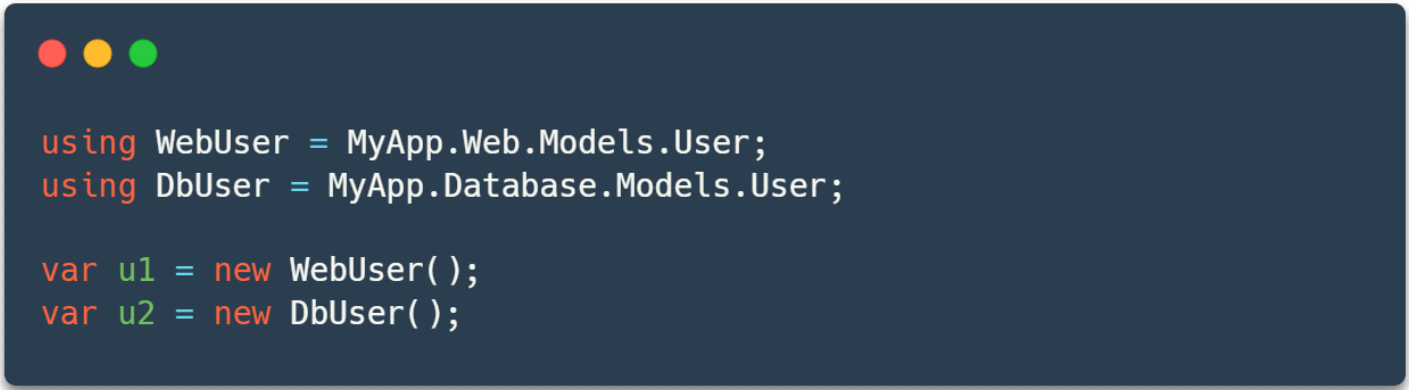
Użycie – wariant B (polecany)

```
using MyApp.Core.Services.Logging;

var log = new Logger();
```

Aliasy

Jeśli dwie przestrzenie zawierają klasy o tej samej nazwie, można użyć **aliasu**.

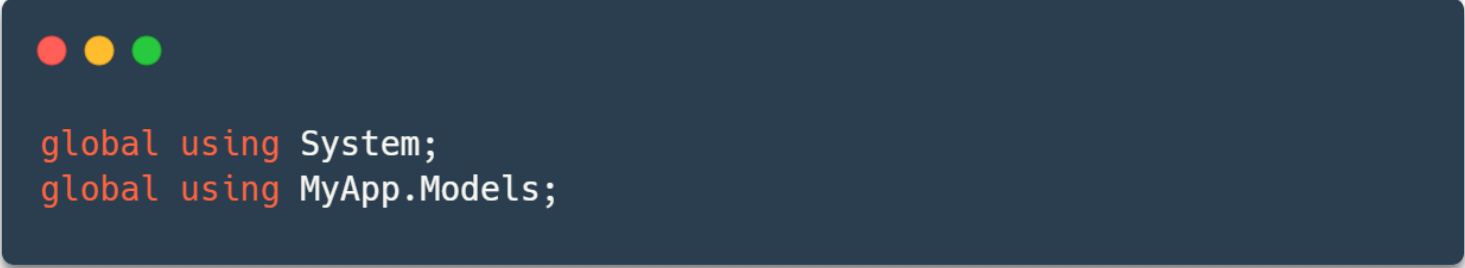


```
using WebUser = MyApp.Web.Models.User;  
using DbUser = MyApp.Database.Models.User;  
  
var u1 = new WebUser( );  
var u2 = new DbUser( );
```

Dzięki aliasom nie ma konfliktu nazw.

Importowanie globalne

Od C# 10 możesz zadeklarować **globalne przestrzenie nazw** (np. w pliku `GlobalUsings.cs`).



```
global using System;  
global using MyApp.Models;
```

Te using są widoczne **we wszystkich plikach projektu**, więc nie trzeba ich powtarzać w każdym pliku.

Konwencje

Formatowanie

- Nawiasy klamrowe { } piszemy w nowej linii.
- Jeden plik = jedna klasa

Wzorce nazewnictwa

Nazwa konwencji	Opis	Przykład
PascalCase ★	Każdy wyraz w nazwie zaczyna się wielką literą, a wszystkie wyrazy są pisane łącznie.	NazwaMojejKlasy, BackColor
camelCase ★	Pierwszy wyraz zaczyna się małą literą, a każdy kolejny wielką. Nazwa przypomina garby wielbłąda.	nazwaMojejZmiennej, isPayment
snake_case	Wszystkie wyrazy pisane są małymi literami i oddzielone od siebie znakiem podkreślenia _.	nazwa_mojej_zmiennej, user_id
kebab-case	Wszystkie wyrazy pisane są małymi literami i oddzielone od siebie myślnikiem -.	nazwa-mojego-stylu, api-endpoint
SCREAMING_SNAKE_CASE	Wszystkie wyrazy pisane wielkimi literami, oddzielone od siebie znakiem podkreślenia _.	LICZBA_PI, ADRES_SERVERA

Tabela przedstawiająca powszechnie obowiązujące wzorce nazewnictwa.
W C# używamy głównie **PascalCase** oraz **camelCase**, ale także **SCREAMING_SNAKE_CASE**.

Wzorce nazewnictwa

Zastosowanie	Styl	Przykład
Klasa	PascalCase	<code>class MojaKlasa { }</code>
Właściwość	PascalCase	<code>public int Wiek { get; set; }</code>
Metoda	PascalCase	<code>public int WypiszTekst() { }</code>
Przestrzeń nazw	PascalCase	<code>MojaAplikacja.Web.Models</code>
Zmienne	camelCase	<code>var licznik = 0;</code>
Parametry metod	camelCase	<code>void Dodaj(int a, int b) { }</code>
Pola prywatne	<code>_camelCase</code>	<code>private readonly DbContext _db;</code>
Stałe	SCREAMING_SNAKE_CASE	<code>const int WERSJA = 2;</code>

Tabela przedstawiająca obowiązujące konwencje, dla elementów w C#, które dotąd poznaliśmy.

Pozostałe zasady

- Kod starajmy się pisać w języku angielskim (*tak się po prostu ogólnie przyjęło*).
- Komentarze teoretycznie też powinny być po angielsku, jednak jeżeli zespół składa się tylko z osób polskojęzycznych, często można korzystać z polskich komentarzy.
- Starajmy się nazywać klasy, metody i zmienne tak, aby po samej nazwie możliwe było wyciągnięcie wniosków co do zastosowania.
- Nazwy katalogów w projekcie (oraz samego projektu) z wielkiej litery – PascalCase.